

Synopsis – Spark V3 – formation avancée

Parcours : Système de traitement de la Data

Niveau d'expertise : Intermédiaire / Expert

Formation : SPARK V3 - FORMATION AVANCEE

Modalités : Présentiel – Distanciel

Durée : 3 jours consécutifs – 21 heures

Objectif global du module :

Cette formation vous permettra d'acquérir une maîtrise avancée de Spark, de comprendre des concepts fondamentaux à la maîtrise du développement Spark en production. Vous serez en mesure de manipuler des données avec Spark SQL, d'optimiser les performances de vos applications Spark, de surveiller et de gérer vos jobs Spark avec les outils appropriés, et enfin, de développer des applications Spark en temps réel pour le traitement de flux de données en production.

Objectifs détaillés :

- Comprendre les concepts de base de Spark.
- Utiliser Spark Shell pour interagir avec Spark.
- Maîtriser le développement Spark en production.
- Acquérir des compétences en Spark SQL.
- Exploiter Spark SQL pour la manipulation de données tabulaires.
- Monitorer et gérer des applications Spark.
- Optimiser les performances de Spark.
- Approfondir les optimisations de performances de Spark.
- Comprendre Spark Delta Lake et son intégration avec Databricks.
- Utiliser Spark Streaming pour le traitement de flux de données en temps réel.
- Développer des applications Spark en temps réel.
-

Séquences			Sous-séquences	Contenu	Durée	Approche	Méthodes & outils	Evaluation
Jour 1								
	Matin	1	Introduction à Spark					
		1.1	Introduction à Spark		90 mn			
			Comprendre les concepts fondamentaux de Spark, y compris son architecture, et ses composants clés	Historique de Spark, Architecture logique, distribution, composants, Drivers, Exécuteurs, RDD, Dataset, Dataframes, Actions et Transformations, RDD Lineage. Explication des notions de résilience, et de distribution. Différence entre les API RDD, Dataset, Dataframes. Anatomie d'une application Spark en détail : Définitions précises des composants essentiels de Spark : (Driver, Executor, Jobs, stages, tasks) Explication de la notion de closure.		Expositive, participative	Présentation Théorique	
		1.1	Prise en main Spark Shell		90 mn			
			Maîtriser l'utilisation de Spark Shell pour interagir avec Spark, via l'exécution de commandes Spark, et en explorant les opérations de base sur RDD et DataFrames.	• Connexion SSH à l'instance master • Upload des données sur HDFS • Exécution de Spark sur gitpod/docker • Chargement d'un fichier CSV dans un DataFrame Spark • Lancement d'une console PySpark / Scala • Activités dans la console Shell • Exécution de commandes Spark • Création et affichage de RDD • Opérations sur les RDD : prise d'éléments, comptage, etc • Exploration de l'immutabilité et de la transformation des RDD • Jointure et tri des RDD • Débogage des RDD avec toString() • références à Java et Scala dans Spark • Opérations sur les DataFrame : • Création d'un DataFrame à partir d'un fichier CSV • Comptage des entrées dans un DataFrame • Filtrage et regroupement des données dans un DataFrame • Tri et requêtes conditionnelles dans un DataFrame • Ressources utiles : • Référence de l'API Spark DataFrame		Expositive, participative	Utiliser Spark Shell pour créer un workflow simple avec chargement de données, manipulations de RDD et DataFrames, suivi d'exécutions de requêtes sur un DataFrame, puis explorer les ressources de référence.	

	Après-Midi	2	Spark Dev + SQL				
		2.1	Développement Spark en production		60 mn		
			Maîtriser le développement Spark en production, de l'initialisation du projet à l'exécution avec spark-submit.	Initialiser un projet Spark Scala avec sbt • Créer le fichier scala • Rédaction du code • Import de SparkSession • Initialisation de SparkSession • Lecture d'un fichier CSV depuis HDFS • Afficher les données avec df.show() et df.filter() • Dépendances dans build sbt: Définition du nom, de la version de Scala, et les dépendances Spark (spark-sql) • Compilation, Utilisation de sbt package dans le répertoire du projet • Generation de l'archive jar • Exécution : Lancement de l'application avec spark-submit		Expositive, participative	Créer un projet Spark Scala avec sbt, lire un fichier CSV depuis HDFS, compiler, générer une archive jar, et lancer l'application avec spark-submit
		2.2	Introduction à Spark SQL		60 mn		
			Acquérir les compétences de Spark SQL pour manipuler des données tabulaires, créer des tables, utiliser SQL pour les transformations et les requêtes, et optimiser les performances.	Spark SQL : Utilisation de SQL DataFrame pour représenter les données de manière tabulaire. • Chargement de données : Import depuis des sources via DataFrameReader.read() pour des formats comme CSV, bases de données, etc. • Tables persistantes avec saveAsTable() • Tables temporaires avec createOrReplaceTempView() • Transformations et syntaxe SQL : Sélection (select), regroupement (groupBy), et tri (orderBy). • Requêtes SQL : Agrégations avec agg , jointures avec join • Requêtes SQL complexes, imbriquées, et composées • Optimisation des performances : Utilisation de DataFrame.repartition() , DataFrame.coalesce() , réglage de spark.memory.fraction , DataFrame.cache()		Expositive, participative	Pratiquer le chargement de données, la création de tables, les opérations SQL de sélection, de regroupement et de tri, les requêtes avancées et l'optimisation des performances avec Spark SQL.

	2. 3	Exploitation de Spark SQL		60 mn			
		Exploiter Spark SQL pour la lecture de fichiers sources, l'écriture de données au format Parquet, l'inférence du schéma, l'exploration des données, les jointures, la création de vues temporaires, et l'analyse de données avec des fonctions SQL.	Lecture de fichiers CSV depuis S3 • Écriture des données dans un format Parquet • Lecture de fichiers Parquet. Enregistrement sous forme de table • Inférence du schéma des données avec inferSchema • Exploration des données : Affichage des colonnes, schéma, statistiques usuelles • Utilisation de jointures pour combiner différentes tables de données • Création et utilisation d'une vue temporaire pour simplifier l'accès aux données et effectuer des requêtes plus complexes • Calcul du taux d'annulation des vols • Utilisation de fonctions SQL telles que "count", "distinct" et "group by" pour agréger et analyser les données • Affichage des résultats sous forme de tableaux et de graphiques pour visualiser les tendances et les statistiques usuelles		Expositive, participative	Lire des fichiers CSV depuis S3, écrire des données au format Parquet, explorer les données, effectuer des jointures, créer et utiliser une vue temporaire, utiliser des fonctions SQL pour l'agrégation et l'analyse, puis visualiser les résultats.	

Jour 2								
	Matin	3	Spark Advanced / Internals					
		3.1	Monitoring : Yarn UI		60 mn			
			Comprendre l'utilisation de l'interface utilisateur YARN (Yarn UI) pour surveiller et gérer les applications Spark en cours d'exécution	• Vue d'ensemble des Applications : ID de l'application, nom, utilisateur, queue, état (ex., RUNNING, FINISHED), progression, type d'application (ex., SPARK), temps de démarrage et de fin, utilisation de la mémoire et du CPU. • Détails de l'Application : Configuration, détails sur les tentatives d'exécution, liens vers les logs, détails sur les exécuteurs Spark. • Utilisation des Ressources : Graphiques de l'utilisation de la mémoire, du CPU, utilisation des ressources par nœud. • Logs d'Application : Logs des containers, des exécuteurs, erreurs et avertissements. • Monitoring en temps réel : État actuel de chaque job, statistiques de performance en temps réel, alertes et notifications. • Historique des Jobs : Liste des jobs terminés, durée d'exécution, ressources utilisées, statut de fin (réussi, échoué). • Diagnostic : Messages d'erreur détaillés, informations sur les causes d'échec, recommandations pour le dépannage. • Killing de Jobs : Bouton ou commande pour arrêter un job, confirmation de l'arrêt, statut après arrêt.		Expositive, participative	Explorer l'interface YARN UI pour visualiser les détails des applications Spark, surveiller l'utilisation des ressources, accéder aux logs, et comprendre les statistiques de performance en temps réel.	

	3.2	Monitoring : Spark UI Deep Dive		60 mn			
		Explorer en profondeur l'interface utilisateur Spark UI pour surveiller et analyser en détail les performances des applications Spark.	Jobs : Liste des jobs avec ID, statut, durée; détails par job incluant temps d'exécution, étapes (stages), tâches. • Stages : Liste des stages avec ID et statut, que nombre de tâches, durée, données lues/écrites. • Storage : Détails des RDDs stockés avec taille, mémoire disque utilisée, partitions. • Environment : Détails de la configuration Spark , variables d'environnement et propriétés JVM . • Executors : Liste des exécuteurs avec ID, adresse; statistiques : temps de tâche, mémoire utilisée, GC time . • SQL : Historique des requêtes SQL; plan d'exécution , statistiques de performance . • Streaming : Détails des batches de streaming incluant durée, données traitées, graphique de débit et latence. • RDDs : Liste et dépendances des RDDs; métriques de taille et de partitionnement . • Scheduler Delay : Temps d'attente avant l'exécution d'une tâche. • JVM GC : Temps passé en garbage collection par les exécuteurs. • Shuffle Read/Write Metrics : Données lues et écrites pendant le shuffle. • Broadcast Variables : Taille et statistiques des variables diffusées. • Accumulators : Valeurs actuelles pour le débogage. • Event Timeline : Chronologie des événements de jobs et stages. • Thread Dump : Informations de débogage pour les threads des exécuteurs.		Expositive, participative	Utiliser Spark UI pour examiner les jobs, les stages, la gestion de la mémoire, la configuration, les exécuteurs, les requêtes SQL, les métriques de streaming, les RDD, et d'autres statistiques de performance dans le contexte d'une application Spark en cours d'exécution.	

		3.3	Paramètres et modalités d'exécution de programmes Spark		60 mn			
			Comprendre les différents modes de déploiement de Spark, les commandes et options générales, ainsi que les paramètres spécifiques à YARN et les options avancées pour exécuter des programmes Spark.	Explication détaillée des Modes de déploiement: cluster, client, local Commandes et Options Générales : --class : Nom de la classe principale (entry point) de l'application. --master : Spécifie le gestionnaire de cluster (ex., yarn, local[4]) --deploy-mode : Mode de déploiement (client ou cluster). --conf : Paramètres de configuration Spark (ex., --conf spark.executor.memory=2g). --packages : Spécifie les dépendances Maven à télécharger.--jars : JARs supplémentaires à utiliser dans le job.--files : Fichiers à copier sur les nœuds exécuteurs. Options YARN : --num-executors : Nombre d'exécuteurs pour le job. --executor-cores : Nombre de cœurs par exécuteur. --executor-memory : Quantité de mémoire par exécuteur. --queue : Nom de la file d'attente YARN à utiliser Options Avancées : --driver-memory : Mémoire pour le processus driver.--driver-cores : Nombre de cœurs pour le driver.--total-executor-cores : Nombre total de cœurs pour tous les exécuteurs.		Expositive, participative	Déploiement d'une application Spark en utilisant différents modes (cluster, client, local), en définissant des paramètres de configuration, des dépendances et les options avancées pour gérer efficacement les ressources et l'exécution de l'application.	

	Après-Midi	4	Optimisation Spark Avancée					
		4.1	Optimisations Spark - Part 1		60 mn			
			Comprendre les optimisations de performance de Spark , notamment la gestion de la mémoire, le partitionnement des données, l'optimisation des requêtes SQL et la gestion des ressources.	• Gestion de la Mémoire : Ajuster <code>spark.executor.memory</code> et <code>spark.driver.memory</code>, Optimiser la sérialisation avec <code>spark.serializer</code> • Partitionnement de Données : Ajuster <code>spark.default.parallelism</code> et <code>spark.sql.shuffle.partitions</code>, Utiliser <code>repartition()</code> ou <code>coalesce()</code> pour optimiser les partitions • Optimisation des Requêtes SQL : Utiliser des DataFrames/Datasets pour tirer parti de Catalyst Optimizer, Utiliser des fonctions intégrées pour les transformations • Gestion des Ressources : Configurer <code>spark.executor.cores</code> et <code>spark.task.cpus</code>, Utiliser des gestionnaires de ressources efficaces (YARN, Mesos)		Expositive, participative	Appliquer les techniques d'optimisation de Spark, y compris l'ajustement de la mémoire, la configuration du partitionnement des données, l'optimisation des requêtes SQL, et la gestion efficace des ressources en utilisant des gestionnaires comme YARN Kubernetes ou Mesos.	
		4.2	Optimisations Spark - Part 2		60 mn			
			Approfondir les optimisations de performance de Spark , en se concentrant sur la mise en cache, l'optimisation des opérations de shuffle, l'optimisation des jointures, le réglage des jobs et des stages, ainsi que le monitoring et le profiling.	• Caching et Persistance : Utiliser <code>persist()</code> ou <code>cache()</code> pour les RDDs/Datasets fréquemment accédés, Choisir le bon niveau de persistance • Optimisation des Shuffles : Minimiser les opérations de shuffle (ex., <code>reduceByKey</code> au lieu de <code>groupByKey</code>), Ajuster <code>spark.shuffle.compress</code> et <code>spark.shuffle.file.buffer</code> • Optimisation du Garbage Collector (GC) : Utiliser le GC Concurrent pour minimiser les pauses, Ajuster les options JVM pour GC • Broadcast et Accumulateurs : Utiliser des variables broadcast pour les grandes données de référence, Utiliser des accumulateurs pour les agrégats globaux • Optimisation des Joins : Préférer les broadcast		Expositive, participative	Mettre en pratique les techniques d'optimisation de Spark, notamment la mise en cache des données, la minimisation des opérations de shuffle, l'ajustement du Garbage Collector, et l'utilisation de Spark UI pour le monitoring en temps réel et le profiling de l'application.	

				join pour les petites tables, Éviter les cartesian join • Tuning des Jobs et des Stages : Analyser les DAGs pour optimiser les stages, Ajuster la granularité des tâches, Monitoring et Profiling : Utiliser Spark UI pour le monitoring en temps réel, Profiler l'application pour identifier les goulots d'étranglement.				
			Spark Delta Lake		60 mn			
			Comprendre les principes et avantages de Spark Delta Lake, et l'intégration avec Databricks.	Principe de fonctionnement • ACID Transactions : Support des transactions atomiques pour la cohérence des données • Schema Enforcement & Evolution : Contrôle strict du schéma et possibilité d'évolution • Time Travel (Versioning) • Accès aux versions antérieures des données • Scaling Metastore : Gestion améliorée des métadonnées pour grands ensembles de données • Upserts et Deletes : Facilité de mises à jour et suppressions • Indexing et Z-Ordering : Optimisations pour accélérer les requêtes • Unification des Batch et Streaming : Traitement unifié des données en batch et en streaming • Optimisation de Lecture/Ecriture : Performances accrues pour les opérations de lecture et d'écriture • Intégration avec Databricks • Support des Formats de Fichier : Compatible avec Parquet et autres formats • Conversions de Table : Conversion facile de tables existantes en tables Delta.		Expositive, participative	Utiliser Spark Delta Lake pour créer une table avec prise en charge ACID, effectuer des opérations d'écriture, de lecture, de mise à jour et de suppression, et explorer les fonctionnalités avancées telles que le contrôle du schéma, l'accès aux versions antérieures et les optimisations de requêtes.	

Jour 3								
	Matin	5	Spark Streaming					
		5.1	Spark Streaming - Part 1		90 mn			
			Comprendre Spark Streaming et son utilisation pour le traitement de flux de données en temps réel.	Streaming en temps réel avec Spark Structured Streaming • Traitement de flux de données avec Streaming Dataframe • Lecture de Sources de données telles que Kafka ou des sockets avec readStream • Transformation de flux avec des opérations comme select, filter, groupBy , etc. • Fenêtres temporelles avec window • Agrégations en temps réel avec agg ou groupBy • Jointures de flux avec join • Écriture de résultats en streaming avec writeStream • Gestion des erreurs et des retards avec checkpoint et watermark • Traitement par lots en streaming avec foreachBatch • Intégration avec d'autres outils et services tels que Kafka, HBase, ou Cassandra. • Utilisation de writeStream pour écrire les résultats dans des formats comme Parquet , JSON, ou Kafka • Configuration des options de sortie : • Format de sortie avec format • Mode de sortie avec outputMode		Expositive, participative	Explorer Spark Streaming en implémentant le traitement de flux de données en temps réel avec des opérations sur les données, la gestion des fenêtres temporelles et de l'écriture de résultats en streaming.	
		5.2	Spark Streaming - Part 2		90 mn			
			Approfondir la maîtrise de Spark Streaming en explorant les accumulateurs, les options de streaming, la gestion de la tolérance aux pannes, et l'optimisation des performances.	Accumulateurs et broadcast de Variables • Options de Streaming • Nom de la requête avec queryName • Fréquence de traitement avec trigger • Partitionnement des résultats avec partitionBy • Chemin de destination avec path • Emplacement des métadonnées avec checkpointLocation • Gestion de la tolérance aux pannes avec failOnDataLoss • Contrôle de la fréquence de traitement avec trigger • Réception des données en streaming avec awaitTermination • Utilisation de requêtes SQL sur les données en		Expositive, participative	Utiliser des accumulateurs, configurer les options de streaming, gérer la tolérance aux pannes, et optimiser les performances avec Spark Streaming.	

				streaming avec createOrReplaceTempView • Optimisation des performances avec repartition et coalesce				
--	--	--	--	--	--	--	--	--

	Après-Midi	6	Développement d'une application Spark				
		6.1	Réalisation d'une application complète End to End - Partie 1		90 mn		
			Comprendre comment développer une application complète en temps réel avec Spark en production	Connexion à une API Financière • Extraction des données financières en temps réel • Envoi des Données vers Kafka • Créer un Producteur Kafka avec .DataStreamWriter • Utilisation de writeStream pour publier les données • Consommation des Données depuis Kafka • Configurer un Consommateur Kafka avec Spark DataStreamReader • Utilisation de readStream pour lire les données • Affichage en temps réel des données financières • Affichage et traitement en temps réel		Expositive, participative	Développer une application en temps réel qui se connecte à une API financière, extrait des données en continu, les envoie vers Kafka, puis les consomme avec Spark pour un affichage et un traitement en temps réel.
		1.1	Réalisation d'une application complète End to End - Partie 2		90 mn		
			Acquérir les compétences pour agréger des indicateurs financiers en temps réel avec Spark Streaming en utilisant des opérations de Spark Structured Streaming, la génération d'indicateurs et Spark SQL pour des calculs avancés.	Agrégation des Indicateurs Financiers avec Spark Streaming • Utiliser des opérations Spark Structured Streaming (comme groupBy, agg, window) pour calculer des agrégations • Génération d'indicateurs financiers • Utilisation de Spark SQL pour des calculs avancés sur les données agrégées • Architecture LAMBDA/KAPPA : Intégrer données en streaming et batch pour une vue complète		Expositive, participative	Développer une application qui agrège en temps réel des indicateurs financiers avec Spark Streaming, en utilisant des opérations telles que groupBy, agg et window. Appliquez également des calculs avancés avec Spark SQL et explorez l'architecture LAMBDA/KAPPA pour une vue complète des données.

